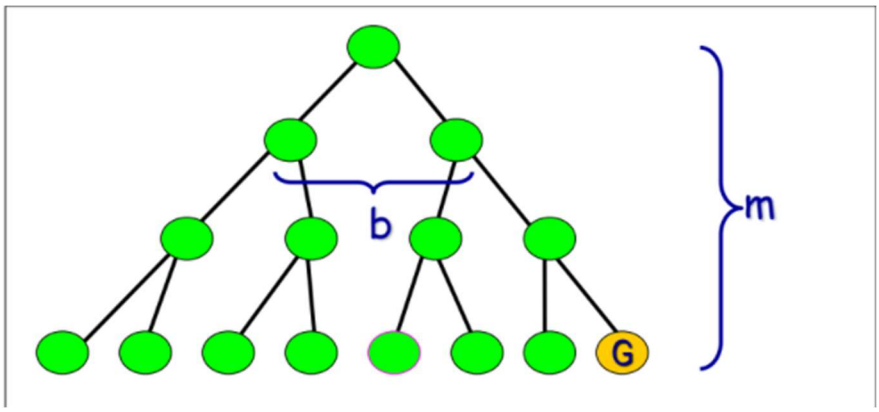
1. Giải quyết vấn đề bằng tìm kiếm:

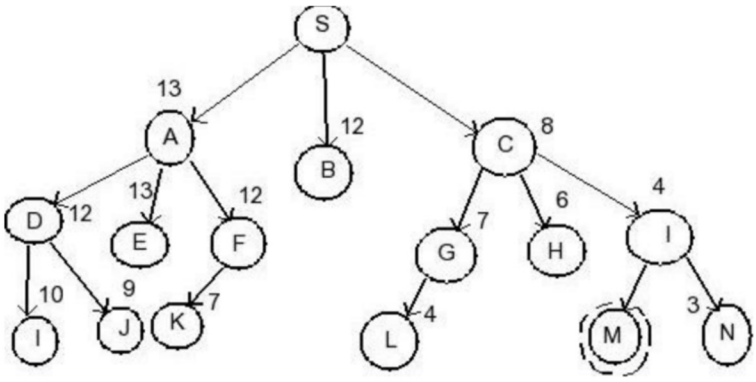
- Các tiêu chính đánh giá:

- độ hoàn chỉnh (Completeness): Có phải lúc nào cũng tìm ra nghiệm nếu có không?
- độ phức tạp về thời gian (Time complexity): số lượng các trạng thái đã đến
- độ phức tạp về không gian (Space complexity): số lượng các trạng thái tối đa phải lưu trong bộ nhớ
- độ tối ưu (Optimality): có phải lúc nào cũng tìm ra nghiệm có chi phí nhỏ nhất không?

- Tìm kiếm mù, DFS: ko hoàn thiện, ko tối ưu, thời gian O(b^d), không gian O(m * b)



- Tìm kiếm mù, BFS: hoàn thiện, tối ưu, không thời gian đều O(b^d) với d là chiều cao của cây.
- Tìm kiếm mù, Tìm kiếm chiều sâu có giới hạn: đi đến độ cao h thì quay lại.
- Tìm kiếm mù, Uniform-Cost Search: Tính d[i] là khoảng cách từ đỉnh i đến đỉnh gốc. Duyệt theo priority queue, ưu tiên theo d[i].
- Tìm kiếm kinh nghiệm, hay có định hướng là phương pháp sử dụng hàm đánh giá làm kim chỉ nam tìm kiếm, giúp tiết kiệm không thời gian.
- Tìm kiếm có định hướng, Best-First Search:
    - Dùng priority queue, ưu tiên hàm nào có h(u) min. h(u) là hàm đánh giá giá trị của đỉnh u tìm kiếm thấy đích.
    - Thuật toán có thể bị quẩn, nên không chắc chắn tìm thấy đích(không hoàn thiện). Không tối ưu, độ phức tạp thời gian, không gian xấu nhất đều là O(b^d) nếu viết hàm đánh giá gà.
- Tìm kiếm có định hướng, Beam:



- Mỗi tầng phát triển đúng W đỉnh. Chọn W đỉnh phát triển dựa vào hàm đánh giá.
- Vì vứt nhiều nhánh đi không phát triển, nên có thể không tìm thấy đích, tối ưu không. Time: O(W * b * m). Space: O(W).

- Tìm kiếm có định hướng, tìm kiếm leo đồi: Như dfs, đi sâu xuống tới

cùng. Nhưng mỗi khi chọn các đỉnh con để phát triển, thay vì duyệt từ trái sang phải, ta sort lại theo hàm đánh giá rồi phát triển đỉnh có hàm đánh giá tốt nhất. DFS là thuật toán đi cắn bừa thì leo đồi là thuật đi cắn có định hướng.

- **Tìm kiếm có định hướng, A*:**

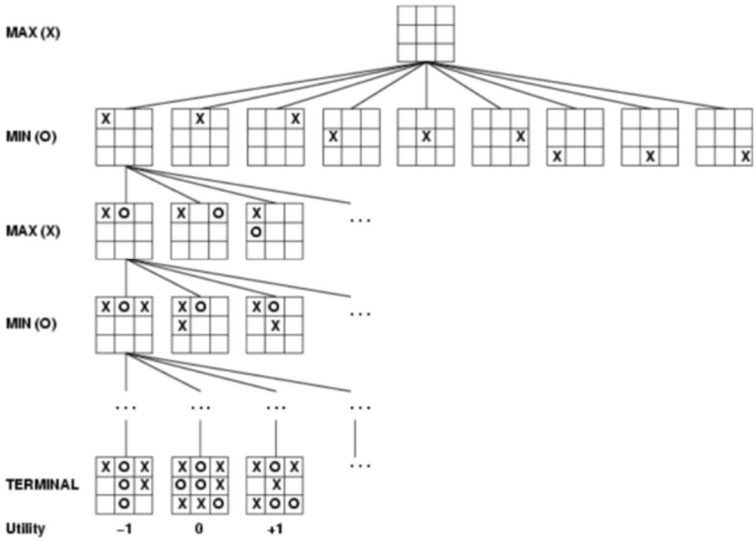
- A* sử dụng hàm $f(u) = g(u) + h(u)$
  - $g(u)$: tổng chi phí từ xuất phát đến u
  - $h(u)$: ước lượng chi phí từ u đến đích.
  - $f(u)$: ước lượng tổng chi phí của phương án khi qua u.
- Tìm kiếm tốt nhất đầu tiên → sử dụng $h(u)$
  ➔ Hiệu quả nhưng không tối ưu
- Tìm kiếm theo chiều rộng → sử dụng $g(u)$
  ➔Tối ưu nhưng không hiệu quả

Lược đồ thuật toán A*

1. Khởi tạo danh sách L chỉ chứa trạng thái ban đầu
2. **Loop do**
   - 2.1 **If** (L rỗng) **then**
     {thông báo tìm kiếm thất bại; kết thúc}
   - 2.2 Loại trạng thái u ở đầu danh sách L;
   - 2.3 **If** (u là trạng thái kết thúc) **then**
     {thông báo tìm kiếm thành công; kết thúc}
   - 2.4 **For** (mỗi trạng thái v kề u) **do**
     {g(v) := g(u) + k(u,v);
     f(v) := g(v) + h(v);
     Đặt v vào danh sách L sao cho L được sắp xếp thep thứ tự tốt đến xấu của hàm đánh giá f;}

- **ĐIỀU KIỆN ĐỂ A* TỐI ƯU VÀ COMPLETE(HOÀN THIỆN) LÀ TOÀN BỘ HÀM ĐÁNH GIÁ NHỎ HƠN GIÁ TRỊ THỰC TẾ**
- **A* có trọng số: Nhân hàm đánh giá với alpha > 1. Sẽ ít trạng thái được phát triển hơn tìm thấy đích nhanh hơn nhưng đường đi không phải ngắn nhất.(chênh lệch kết quả tối ưu không quá alpha lần).**

- **Tìm kiếm có đối thủ: Minimax, cắt cụt alpha-beta:**



- **Vấn đề của bên trắng là tìm ra chuỗi nước đi sao cho xen kẽ với các nước đi của đen tạo thành đường đi từ trạng thái ban đầu tới trạng thái kết thúc là cao nhất (ta phải liệt kê cả nước đi của đối thủ).**
- **Để thuận lợi thì biểu diễn theo cây trò chơi. Như hình vẽ, đen trắng xen kẽ nhau trong đó ở nước đi tiếp theo của trắng thì ta sẽ lấy max, tương tự nước của đen lấy min. (trường hợp đang nói đây là thắng đc 1, hòa 0, thua -1)**
- **Chiến lược minimax là chiến lược tìm kiếm theo độ sâu và sử dụng hàm đánh giá để giảm bớt cây tìm kiếm.**
- **Thuật toán minimax hoàn thiện nếu cây trò chơi không vô hạn. minimax tối ưu, độ phức tạp không gian O(bm) và thời gian O(b^m).**
- **Trong nhiều trò chơi tổng số nước là lớn(như cờ vua, cờ tướng). Độ phức tạp cũng tăng theo m là tầng của cây trò chơi, cũng là số nước đi. Độ phức tạp lớn là trải nghiệm không tốt. Vì vậy, ta chỉ có thể xét đến số tầng nhất định(thường có thể 4,5,6 nước) và ta đánh giá giá trị tiềm năng của nước đi này.**

- **Phương pháp cắt cụt alpha-beta giúp cắt nhánh thừa không cần thiết phát triển.**

Phương pháp cắt cụt alpha-beta

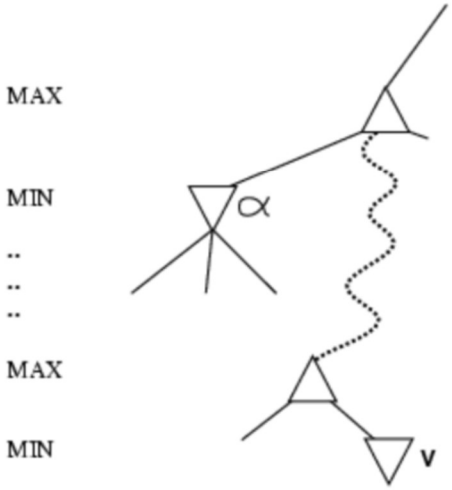
- Cho phép cắt bỏ các nhánh không cần thiết cho sự đánh giá đỉnh u

Nếu a là đỉnh trắng, nếu u, v đã được gán và giá trị của u > giá trị của v thì không cần gán giá trị của a nữa



Tại sao lại gọi là α-β?

- α là giá trị tốt nhất (i.e., giá trị cao nhất) tính đến thời điểm hiện tại cho *max*
- Nếu *v kém hơn* α, *max* sẽ loại bỏ v
  → cắt nhánh
- Định nghĩa β tương tự cho *min*
-



2. Tri thức và lập luận:

- **Tri thức chắc chắn:**

Dạng chuẩn tắc

- Để dễ dàng viết các chương trình máy tính chúng ta chuẩn hóa các công thức đưa chúng về ***dạng chuẩn hội***, tức là hội của các câu tuyển.
- Áp dụng thủ tục sau :
  - Bỏ các dấu kéo theo →: $A \rightarrow B = \neg A \vee B$
  - Chuyển các dấu phủ định ¬ vào sát các ký hiệu mệnh đề bằng luật De Morgan
  - Áp dụng luật phân phối

$$A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$$

Các luật suy diễn

- Luật suy diễn khẳng định
- Luật suy diễn phủ định
- Luật bắc cầu
- Luật loại bỏ hội
- Luật đưa vào hội
- Luật đưa vào tuyển
- Luật phân giải

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

$$\frac{\alpha \Rightarrow \beta, \neg\beta}{\neg\alpha}$$

$$\frac{\alpha \Rightarrow \beta, \beta \Rightarrow \gamma}{\alpha \Rightarrow \gamma}$$

$$\frac{\alpha 1 \wedge \ldots \wedge \alpha i \wedge \ldots \alpha m}{\alpha i}$$

$$\frac{\alpha 1, \ldots, \alpha i, \ldots, \alpha m}{\alpha 1 \wedge \ldots \wedge \alpha i \wedge \ldots \wedge \ldots \alpha m}$$

$$\frac{\alpha i}{\alpha 1 \vee \ldots \vee \alpha i \vee \ldots \vee \alpha m}$$

$$\frac{\alpha \vee \beta, 7\beta \vee \gamma}{\alpha \vee \gamma}$$

Luật phân giải (Resolution)

- Luật phân giải là sự tổng quát của các luật khẳng định, phụ định và luật bắc cầu.
- Luật phân giải trên các câu tuyển

$$\frac{A_1 \vee A_2 \vee \ldots \vee A_m \vee C \qquad \neg C \vee B_1 \vee B_2 \vee \ldots \vee B_n}{A_1 \vee \ldots \vee A_m \vee B_1 \vee \ldots \vee B_n}$$

Chứng minh bác bỏ bằng luật phân giải

- Ý tưởng:

Từ G muốn chứng minh A, ta chứng minh từ G và]A sẽ suy ra được câu rỗng (hay mâu thuẫn)

Thuật toán phân giải

Tư tưởng

- Hoạt động dựa trên phương pháp chứng minh phản chứng và luật phân giải.
- Lần lượt ghép hai cặp câu với nhau và áp dụng định luật phân giải sinh ra câu mới đưa vào tập tri thức.
- Cứ làm như vậy cho đến khi không sinh ra được câu mới thì thôi.

THUẬT GIẢI ROBINSON

1. Biến đổi tất cả các câu thành dạng CNF
2. Lấy phủ định kết luận, đưa vào KB
3. Lặp
   a. Nếu trong KB có chứa hai mệnh đề phủ định nhau thì trả về ***false //trả về false => vô lý***
   b. Nếu có hai mệnh đề chứa các literal phủ định nhau thì áp dụng hợp giải.
   c. Lặp cho đến khi không thể áp dụng tiếp luật hợp giải.
4. Trả về ***true***

Luật thay thế phổ dụng

- Luật thay thế phổ dụng phát biểu rằng từ công thức $\forall x\ G$ suy ra công thức $\forall x\ G \rightarrow G[x|t]$
  với bất kỳ biến x và thay thế bởi hạng thức t

- Ví dụ: $\forall x\ King(x) \wedge Greedy(x) \Rightarrow Evil(x)$ sinh ra:
  $King(John) \wedge Greedy(John) \Rightarrow Evil(John)$
  $King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard)$
  $King(Father(John)) \wedge Greedy(Father(John)) \Rightarrow Evil(Father(John))$

Chú ý:

- Một hằng số a không được thay bằng một biến x
- Một biến x không được thay bằng một hạng thức có chứa x

- **Tri thức không chắc chắn: Dùng lý thuyết xác suất**

Công thức Bayes

- $$\Pr(X|Y) = \frac{\Pr(X,Y)}{\Pr(Y)} = \frac{\Pr(Y|X)*\Pr(X)}{\Pr(Y)}$$

Biến ngẫu nhiên X độc lập với biến ngẫu nhiên Y nếu:

$$\Pr(X|Y) = \Pr(X)$$

Biến ngẫu nhiên X độc lập với biến ngẫu nhiên Y có điều kiện Z:

$$\Pr(X|Y,Z) = \Pr(X|Z)$$

Mạng xác suất (1)

- Nếu biết tất cả các $\Pr(X_1, X_2, \dots, X_k)$ → tính được tổ hợp bất kì các biến ngẫu nhiên, tính được xác suất có điều kiện bất kì của các biến ngẫu nhiên
- Nhược điểm
  - Không có đủ hết thông tin $\Pr(X_1, X_2, \dots, X_k)$
  - Không lưu trữ được
- Cần xây dựng mô hình thích hợp để biểu diễn tri thức không chắc chắn
  - Giảm bớt các xác suất ban đầu cần biết trước

- Mạng xác suất

→Mô tả đầy đủ tri thức không chắc chắn

→tiết kiệm không gian nhớ

Mạng xác suất (2)

Mạng xác suất là một đồ thị vô hướng không có chu trình thỏa mãn

- Các đỉnh của đồ thị là các biến ngẫu nhiên
- Mỗi cung từ đỉnh X đến đỉnh Y thể hiện sự ảnh hưởng trực tiếp của biến ngẫu nhiên X lên biến ngẫu nhiên Y (Y phụ thuộc trực tiếp vào X). Đỉnh X gọi là cha của Y.
- Tại mỗi đỉnh được cho phân phối xác suất có điều kiện của đỉnh đó khi cho trước các cha của nó (sự ảnh hưởng các cha tác dụng vào nó). Nếu $X_1, X_2, \dots, X_k$ là cha của Y thì cần biết $\Pr(Y|X_1, X_2, \dots, X_k)$

- Biết giá trị các biến ngẫu nhiên trọng mạng (biến bằng chứng) xác định biến hỏi
- Lập luận chuẩn đoán: Lập luận đi từ hậu quả đến nguyên nhân. Bằng chứng là hậu thế của biến hỏi. Ví dụ: Pr(B|L); Bác sĩ cần chuẩn đoán bệnh khi biết triệu chứng, kết quả xét nghiệm
- Lập luận tiên đoán: Lập luận đi từ nguyên nhân đến kết quả. Biến bằng chứng là tiền thân của biến hỏi. Ví dụ: Biết có trộm cần tính khả năng Mai gọi; Dự đoán kết quả,…
- Trường hợp tổng quát: Biến bằng chứng vừa là tiền thân vừa là hậu thế của biến hỏi. Ví dụ: Biết không có động đất và Lan gọi, tính khả năng có trộm Pr(B=True|E=false, L=true)=?

• $E = E^+ \cup E^-$ (trong đó E^+ / E^- là tập bằng chứng nằm ở trên/dưới X)

$$\Pr(X|E) = \Pr(X|E^+, E^-) = \frac{\Pr(E^-, X, E^+)}{\Pr(E)}$$

$$= \frac{\Pr(E^- \mid X, E^+)\Pr(X \mid E^+)\Pr(E^+)}{\Pr(E)}$$

E^- độc lập với E^+, do đó $\Pr(E^-|X, E^+) = \Pr(E^-|X)$

$\Pr(E^+)/\Pr(E)$ là hằng số α

$$\Pr(X|E) = \alpha \Pr(E^-|X) \Pr(X|E^+)$$

• $\Pr(X|E^+)$ hỗ trợ nguyên nhân cho X

• $\Pr(E^-|X)$ hỗ trợ chuẩn đoán cho X

$$\Pr(X|E^+) = \sum_z \Pr(X|Z = z) \Pr(Z = z|E^+)$$

• $\Pr(X|Z = z)$ đã biết (giá trị được xác định khi xây dựng mạng)

• $\Pr(Z = z|E^+)$ được tính đệ quy

- P(E trừ | X) thì ngồi khóc.
- Mạng đa kết nối: gộp đỉnh

3. Machine Learning:

- Học máy cơ bản

Học máy và AI

- Giải quyết vấn đề bằng suy diễn với thông tin được cung cấp trước → không cải thiện được hiệu quả trong tương lai dựa trên kinh nghiệm thu được
- Cải tiến hiệu quả giải quyết bài toán thông qua quan sát và sự dạy bảo
- Học máy như là một thành phần quan trọng của AI

- Tại sao học máy lại quan trọng: BigData. Xây dựng tri thức để chết à. Không nên quê mùa. Nên phải học máy.

- Biểu diễn một bài toán học máy [Mitchell, 1997]
  - **Một công việc (nhiệm vụ) T**
  - **Đối với các tiêu chí đánh giá hiệu năng P**
  - **Thông qua (sử dụng) kinh nghiệm E**

- **Naive Bayes:**

Phân lớp Bayes

$$p(h \mid d) = \frac{P(d \mid h)P(h)}{P(d)} \qquad h_{MAP} = \arg\max_{h \in H} P(h \mid d)$$

$$h_{MAP} = \arg \max_{h \in H} P(h_j | a_1, a_2, \dots, a_n)$$

$$= \arg \max_{h \in H} \frac{P(a_1, a_2, \dots, a_n | h_j) P(h_j)}{P(a_1, a_2, \dots, a_n)}$$

$$h_{MAP} = \arg \max_{h \in H} P(a_1, a_2, \dots, a_n | h_j) P(h_j)$$

Dữ liệu ***d*** của chúng ta có một vài thuộc tính

Giả sử Bayes ngây thơ: Các thuộc tính mà mô tả dữ liệu là độc lập điều kiện theo bởi giả thuyết phân lớp:

$$P(\mathbf{d} \mid h) = P(a_1, \dots, a_T \mid h) = \prod_t P(a_t \mid h)$$

- Đây là giả sử ngây thơ (hồn nhiên), vì vậy thường trái với thực tế.
- Mặc dù vậy, nó làm việc tốt trong các ứng dụng thực tế

- **SVM**

- SVM là phương pháp phân lớp tuyến tính bằng cách xác định một siêu phẳng (hyperplane) để phân tách hai lớp của dữ liệu. Ví dụ, lớp có nhãn âm (negative) và lớp có nhãn dương (positive)

- SVM tốt và phù hợp đối với những bài toán phân lớp có không gian biểu diễn thuộc tính lớn (nhiều thuộc tính)

- Đối với một ví dụ x_i:

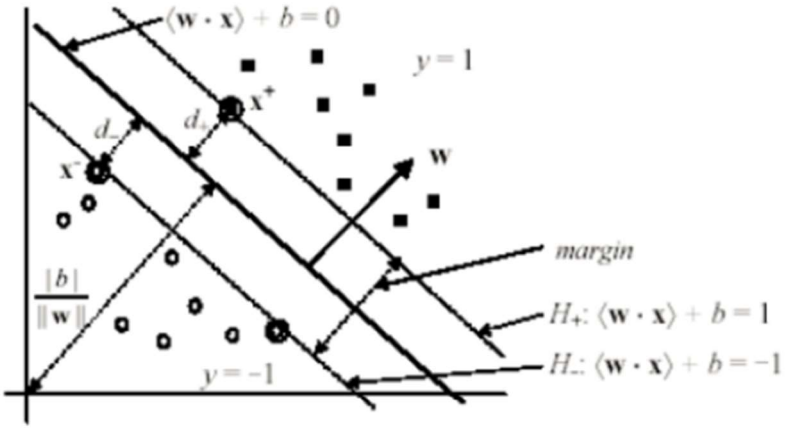
$$y_i = \begin{cases} +1 \, nếu < w \cdot x_i > +b \geq 0 \\ -1 \, nếu < w \cdot x_i > +b < 0 \end{cases}$$

- SVM xác định một hàm phân tách tuyến tính

$$f(x) =< w \cdot x > +b$$

Trong đó w là vector trọng số các thuộc tính; b là giá trị thực

- SVM lựa chọn mặt siêu phẳng phân tách có lề (margin) cực đại
- Lý thuyết học máy đã chỉ ra rằng một mặt siêu phẳng phân tách như thế sẽ tối thiểu hóa giới hạn lỗi (phân lớp) mắc phải.



- Xét một ví dụ các lớp dương $(x^+, 1)$ và ví dụ các lớp âm $(x^-, -1)$ gần nhất đối với siêu phẳng phân tách H_0 $(< w \cdot x > +b = 0)$

H_+ đi qua x^+ và song song với H_0

H_- đi qua x^- và song song với H_0

$$< w \cdot x^+ > +b = \quad 1$$
$$< w \cdot x^- > +b = -1$$
$$< w \cdot x_i > +b \geq \quad 1, nếu \; y_i = +1$$
$$< w \cdot x_i > +b \leq -1, nếu \; y_i = -1$$

- Mức lề là khoảng cách giữa hai siêu phẳng lề H_+ và H_-
  - d_+ là khoảng cách giữa H_+ và H_0
  - d_- là khoảng cách giữa H_- và H_0

$$d_+ = \frac{|\langle \mathbf{w} \cdot \mathbf{x}^+ \rangle + b|}{\|\mathbf{w}\|} = \frac{|1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|} \qquad d_- = \frac{|\langle \mathbf{w} \cdot \mathbf{x}^- \rangle + b|}{\|\mathbf{w}\|} = \frac{|-1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

$$margin = d_+ + d_- = \frac{2}{\|\mathbf{w}\|}$$

- Phương pháp SVM trong trường hợp các ví dụ phân lớp tuyến tính nhưng không thể phân tách được
  - Trường hợp phân lớp tuyến tính và phân tách được là lí tưởng (ít xảy ra)
  - Tập dữ liệu có thể chứa nhiễu lỗi (có thể một số ví dụ được gán nhãn sai)

Nới lỏng các điều kiện

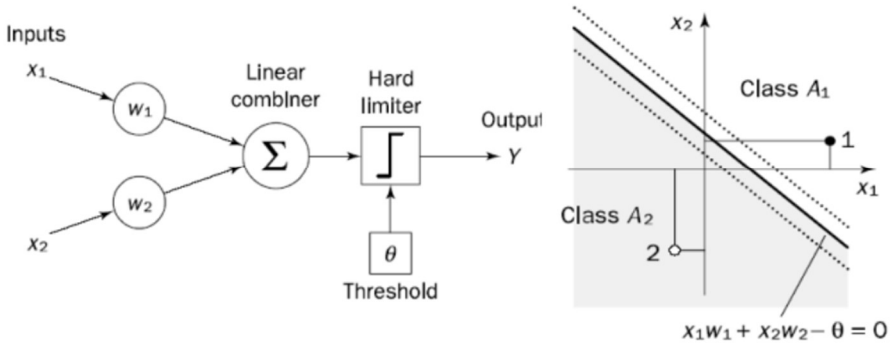
- Để làm việc với các dữ liệu chứa nhiễu, cần nới lỏng các điều kiện lề bằng cách sử dụng các biến slack $\xi_i (\geq 0)$

$$\begin{cases} < w \cdot x_i > +b \geq \quad 1 - \xi_i, nếu\ y_i = +1 \\ < w \cdot x_i > +b \leq -1 + \xi_i, nếu\ y_i = -1 \end{cases}$$

- **Perceptron:**

Perceptron

- Là mạng neuron nhân tạo đơn giản nhất gồm một neuron với các trọng số **có thể điều chỉnh được** và một **giới hạn cứng**.



- Perceptron phân loại các input thành hai lớp A_1 và A_2 bị chia cách bởi một siêu phẳng cho bởi hàm phân biệt tuyến tính

$$\sum_{i=1}^{n} x_i w_i - \theta = 0$$

Perceptron

- Một perceptron học bằng cách điều chỉnh **siêu phẳng phân biệt** theo sai số $e(p) = Y_d(p) - Y(p)$ với $Y_d(p)/Y(p)$ là kết xuất thực/mong muốn tại bước p

- Nếu e(p) > 0 → tăng *Y(p),* ngược lại giảm *Y(p).*
- Nếu $x_i(p) > 0$, tăng $w_i(p)$ sẽ làm tăng Y(p). Ngược lại nếu $x_i(p) < 0$, tăng $w_i(p)$ sẽ làm giảm Y(p).

- Do đó, **luật học perceptron** có thể được viết là

$$w_i(p+1) = w_i(p) + \alpha \times x_i(p) \times e(p)$$

với α là **tốc độ học**

Perceptron – Thuật toán Học

- **Khởi tạo**: $(w_1, w_2, \ldots, w_n, t) \in [-0.5, 0.5]$

- **Kích hoạt**: tính kết xuất thực sự tại lần lặp p = 1

$$Y(p) = step\left[\sum_{i=1}^{n} x_i(p) w_i(p) - \theta\right]$$

- **Cập nhật trọng số:**

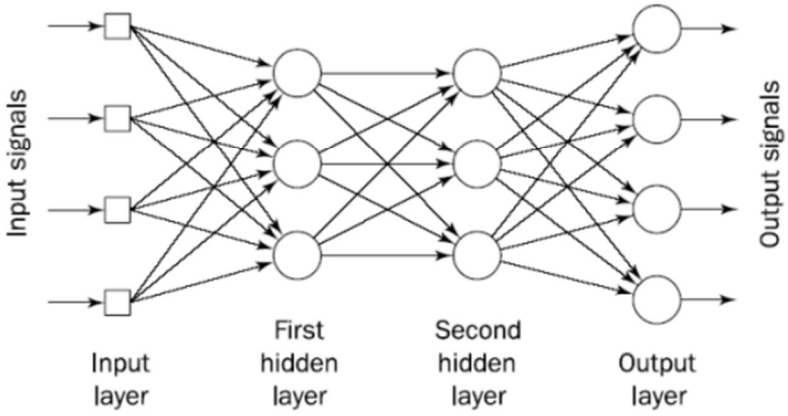
$$w_i(p+1) = w_i(p) + \Delta w_i(p)$$

trong đó $\Delta w_i(p) = \alpha \times x_i(p) \times e(p)$ **(luật học delta)**

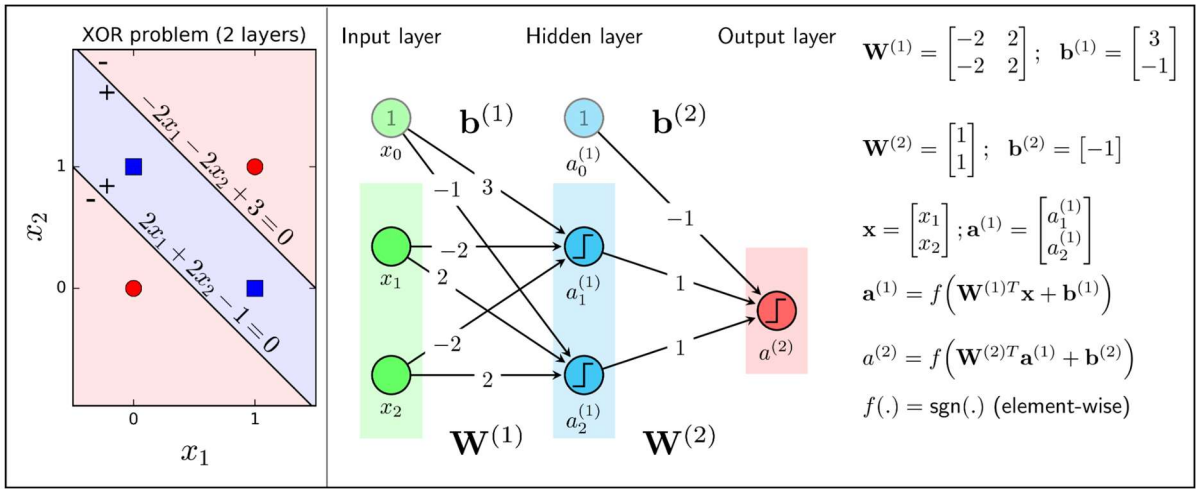
- **Lặp:** tăng p lên 1 và lặp lại bước 2.

- **PERCEPTRON LÀ MẠNG NEURON VỚI 0 LỚP ẨN, NÊN CÓ NHIỀU HẠN CHẾ (NHƯ KHÔNG HỌC ĐƯỢC HÀM XOR).**
- **NEURON**

- Một mạng neuron lan truyền tiến gồm một **lớp nhập**, ít nhất một **lớp ẩn**, và một **lớp xuất**.



- Lớp nhập: nhận các input và phân phối chúng cho tất cả neuron trong lớp ẩn
- Lớp xuất: biểu diễn kết quả của toàn mạng
- Lớp ẩn:
  - Dò tìm các đặc trưng.
  - Các neuron trong lớp này "ẩn" các kết xuất mong muốn của chúng.
  - Mạng một lớp ẩn có thể biểu diễn bất kỳ hàm liên tục nào.
  - Mạng hai lớp ẩn có thể biểu diễn các hàm không liên tục.



Đây là 1 ví dụ phân lớp XOR. Có 1 lớp ẩn với 2 node, mỗi node sẽ cho ra 1 hoặc -1 thông qua xét <w,x> từng node >= 0 hay không. gọi giá trị mỗi node ẩn là z. Ta sẽ lại cập nhật <w',z> tạo ra output. Nếu output khác với kỳ vọng thì sẽ lan truyền ngược lại thay đổi từng w, w'.

Một số câu hỏi lý thuyết ML:

● Naive Bayes:

1. Phát biểu mô hình naive bayes:

- Naive Bayes là mô hình phân lớp học máy dựa trên công thức bayes trong xác suất thống kê.
- Gọi tập d: a1,a2,a3 … an là các thuộc tính. h là các nhãn. Ta có:

$$p(h \mid d) = \frac{P(d \mid h)P(h)}{P(d)} \qquad h_{MAP} = \arg\max_{h \in H} P(h \mid d)$$

$$h_{MAP} = \arg \max_{h \in H} P(h_j | a_1, a_2, \dots, a_n)$$

$$= \arg \max_{h \in H} \frac{P(a_1, a_2, \dots, a_n | h_j) P(h_j)}{P(a_1, a_2, \dots, a_n)}$$

$$h_{MAP} = \arg \max_{h \in H} P(a_1, a_2, \dots, a_n | h_j) P(h_j)$$

- Với mỗi tập dữ liệu d, ta sẽ phân lớp xem thuộc nhãn nào qua h nhãn dựa vào xác suất p(h | d) nào là lớn nhất đối với mỗi nhãn. Việc tính toán P(a1,a2,...,an | h) là khá khó khăn trong thực tế. Vì vậy ta sẽ dùng giả sử bayes ngây thơ, coi các dữ liệu a1,a2,.., là độc lập với nhau.

$$P(\mathbf{d} \mid h) = P(a_1,\dots,a_T \mid h) = \prod_t P(a_t \mid h)$$

- Công việc tính toán trở nên khá dễ dàng. Dù giả sử trên là không đúng thực tế nhưng áp dụng trong thực tế nhưng nó làm việc rất tốt trong thực tế.
- Naive bayes có 1 tập dữ liệu để huấn luyện. Ta chia thành 2 tập là training set và test set. Dùng training set để tính xác suất đưa ra dự đoán nhãn cho test set. Từ đó dùng các mô hình đánh giá hiệu quả của naive bayes.

2. Ứng dụng của naive bayes:

- Chuẩn đoán sức khỏe
- Phân loại thư rác
- Phân loại văn bản

3. Ưu điểm naive bayes:

- Đơn giản, dễ hiểu, dễ cài đặt, tốc độ xử lý nhanh.

- Đòi hỏi các đặc trưng là độc lập, nhưng trong thực tế thì không làm giảm chất lượng của mô hình.

4. **Trường hợp dữ liệu không đầy đủ thì tính xác suất như nào?**

- Có 3 cách, tùy thuộc vào từng trường hợp áp dụng:
    + Loại bỏ bản ghi chứa dữ liệu bị thiếu.
    + Loại bỏ thuộc tính chứa dữ liệu bị thiếu. (không ảnh hưởng đến các thuộc tính khác vì naive bayes tính độc lập các dữ liệu)
    + Ghi lại dữ liệu bị thiếu bằng mean/median/mode hoặc random theo phân bố xác suất của thuộc tính đó.

5. **Dữ liệu là liên tục thì tính xác suất như nào?**

- Dùng gaussian naive bayes. Tính E(Xi), D(Xi) với Xi là thuộc tính liên tục rồi với mỗi giá trị a nhận được ta tính xác suất theo phân bố chuẩn với kỳ vọng và độ lệch chuẩn là E(Xi), D(Xi)

6. **Ngoài ra còn có dùng logarit để giảm sai số hay dùng laplace smoothing để tránh dữ liệu = 0.**

   hình dưới là laplace smoothing, cộng tất cả các dữ liệu với 1. (dùng khi gặp dữ liệu rời rạc)

$$\hat{P}(w_i \mid c) = \frac{count(w_i, c) + 1}{\sum_{w \in V} \left(count(w, c) + 1 \right)}$$

$$= \frac{count(w_i, c) + 1}{\left(\sum_{w \in V} count(w, c) \right) + |V|}$$

● SVM

1. **Phát biểu mô hình SVM:**

- SVM là mô hình xây dựng 1 siêu phẳng hoặc 1 tập hợp các siêu phẳng trong 1 không gian nhiều chiều hoặc vô hạn chiều, có thể được sử dụng cho phân loại, hồi quy.
- SVM chỉ dùng cho phân loại 2 tệp dữ liệu mà có khả năng phân tách tuyến tính.

- SVM là bài toán đi tìm mặt phân cách sao cho *margin* tìm được là lớn nhất, đồng nghĩa với việc các điểm dữ liệu *an toàn nhất* so với mặt phân cách.

• SVM xác định một hàm phân tách tuyến tính

$$f(x) = < w \cdot x > + b$$

Trong đó w là vector trọng số các thuộc tính; b là giá trị thực

$$d_+ = \frac{|\langle \mathbf{w} \cdot \mathbf{x}^+ \rangle + b|}{\|\mathbf{w}\|} = \frac{|1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|} \qquad d_- = \frac{|\langle \mathbf{w} \cdot \mathbf{x}^- \rangle + b|}{\|\mathbf{w}\|} = \frac{|-1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

$$margin = d_+ + d_- = \frac{2}{\|\mathbf{w}\|}$$

đi thi vẽ bừa cái hình ví dụ SVM ra có d+, d-.

- SVM có 1 tập dữ liệu để huấn luyện. Chia thành 2 tập là training set và test set. Dùng training set để tính xác suất đưa ra dự đoán nhãn cho test set. Từ đó dùng các mô hình đánh giá hiệu quả của SVM.

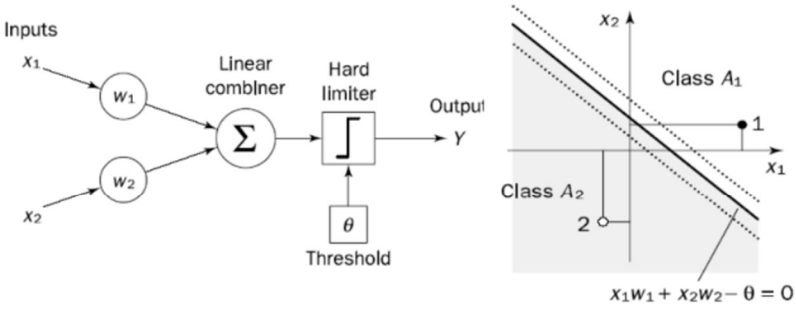
2. Nhược điểm của SVM:

- Chỉ phân được 2 lớp và dữ liệu có khả năng phân tách. Khắc phục như Perceptron ở dưới. Hoặc có thể dùng kernel (thầy không dạy).

● Perceptron

1. Phát biểu mô hình Perceptron.

• Là mạng neuron nhân tạo đơn giản nhất gồm một neuron với các trọng số **có thể điều chỉnh được** và một **giới hạn cứng**.



• Perceptron phân loại các input thành hai lớp A_1 và A_2 bị chia cách bởi một siêu phẳng cho bởi hàm phân biệt tuyến tính

$$\sum_{i=1}^{n} x_i w_i - \theta = 0$$

Trang 42

- Ý tưởng chính: Cứ làm đi, sai đâu sửa đấy, cuối cùng sẽ thành công

- Perceptron là thuật toán classification cho trường hợp đơn giản nhất: chỉ có hai class. Perceptron phân chia 2 tệp dữ liệu dựa trên siêu phẳng.
- Perceptron là thuật toán chạy nhiều thế hệ. Ở mỗi thế hệ, ta sẽ chạy lần lượt mạng neuron nhân tạo với các input đầu vào. Nếu output dự đoán chênh lệch với output kỳ vọng, ta sẽ cập nhật hệ số w theo công thức dưới. Đến khi xuất hiện một thế hệ mà các dự đoán đều đúng thuật toán dừng lại.

Perceptron – Thuật toán Học

• **Khởi tạo**: $(w_1, w_2, \ldots, w_n, t) \in [-0.5, 0.5]$

• **Kích hoạt**: tính kết xuất thực sự tại lần lặp p = 1

$$Y(p) = step\left[\sum_{i=1}^{n} x_i(p)w_i(p) - \theta\right]$$

• **Cập nhật trọng số:**

$$w_i(p+1) = w_i(p) + \Delta w_i(p)$$

trong đó $\Delta w_i(p) = \alpha \times x_i(p) \times e(p)$ **(luật học delta)**

• **Lặp:** tăng p lên 1 và lặp lại bước 2.

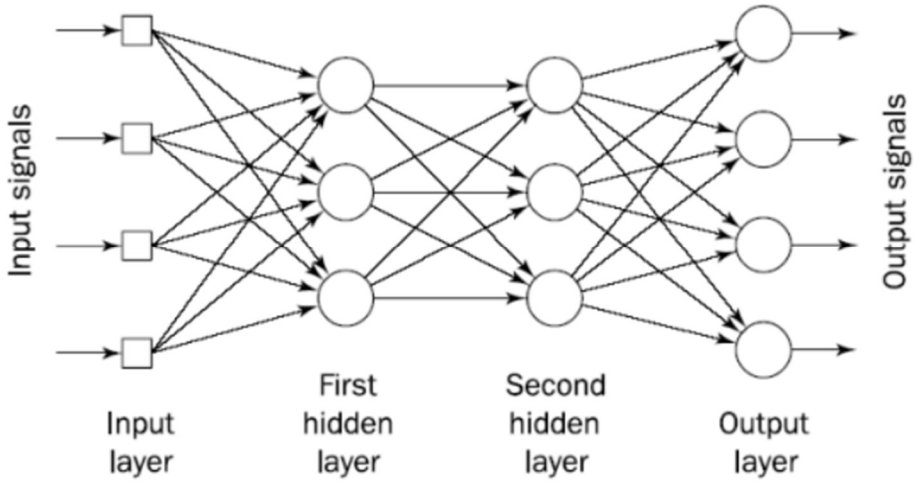
- $e(p) = y_d(p) - y(p)$. $y_d(p)$ là kết output mong muốn. $y(p)$ là output thực.

2. **Giới hạn hay yếu điểm của mô hình Perceptron là gì?**

- Chỉ dùng được hàm có giới hạn cứng(step và sign) và phân loại được 2 lớp. Khắc phục: Dùng neural network.
- Chỉ phân loại được dữ liệu linearly separable(có khả năng phân tách tuyến tính). Nếu dữ liệu không phân tách tuyến tính thì có thể dẫn tới không hội tụ được. Cách khắc phục: Giới hạn số (vòng lặp)thế hệ và lưu lại thế hệ mà phân loại số điểm sai(misclassified) là ít nhất hoặc dùng neural network với nhiều tầng ẩn, mỗi tầng ẩn nhiều node.
- Có vô số các siêu phẳng thỏa mãn và không cho ra nghiệm tối ưu nhất. Khắc phục: có thể dùng SVM.

● Multilayer neural network (mạng neuron nhiều lớp)

1. **Phát biểu mạng mô hình mạng neural nhiều lớp:**



- Mô hình mạng neural là mô hình gồm 3 lớp:
    + Lớp nhập: nhập các input và phân phối chúng ra tất cả các lớp ẩn.
    + Lớp ẩn(có 1 hoặc nhiều lớp): Dò tìm các đặc trưng. (chú ý: 1 lớp ẩn có thể biểu diễn bất kì hàm liên tục. 2 lớp ẩn có thể biểu diễn bất kì hàm không liên tục.)
    + Lớp xuất: biểu diễn kết quả toàn mạng
- Quá trình học gồm 2 bước:
    + Lan truyền input đến các lớp ẩn, tính toán và xuất ra output.
    + Tính toán hàm mất màn từ output thực với output kỳ vọng. Từ đó lan truyền ngược lại để cập nhật trọng số.

2. Ưu điểm nhược điểm của mạng neuron nhiều lớp:

- Ưu điểm:
- Có thể giải quyết được bài toán phức tạp không phân tách tuyến tính.
- Làm việc tốt với dữ liệu lớn.
- Nếu đã training xong thì việc với mỗi input sẽ đưa ra dự đoán nhanh.
- Nhược điểm:
- Nếu nhiều tầng ẩn, và tầng ẩn nhiều node thì tính toán sẽ phức tạp và mất nhiều thời gian.
- Sự chính xác của model phụ thuộc nhiều vào cách chọn tầng ẩn, hàm truyền.

3. Ý nghĩa của tầng ẩn:

- Tầng ẩn thực hiện nhiệm vụ lấy input rồi apply trọng số + dùng hàm truyền để xuất ra output cuối cùng. Tầng ẩn như là 1 "hộp đen" để dự đoán output từ input. Gọi là hộp đen vì tầng ẩn tính toán rất phức tạp với nhiều node cùng với hàm truyền.

- Mỗi tầng ẩn có thể dùng khám phá 1 đặc điểm riêng của dữ liệu (như nhận khuôn mặt thì nhận dạng mắt, mũi, miệng…) từ đó đủ cơ sở để kết luận output chính xác.
- Tầng ẩn giúp việc phân loại dữ liệu giảm bớt hạn chế. Mạng 1 tầng ẩn có thể biểu diễn bất kỳ hàm liên tục nào. Mạng 2 tầng ẩn có thể biểu diễn bất kì hàm không liên tục nào.